Term Project: Hotel Management System

The Hotel Management System project is mainly developed to assist hotel managers and staff in managing and streamlining hotel operations efficiently. The system provides various functionalities, such as making room reservations, guest management, billings, and generating profit reports. The purpose is to build this in a way that it reduces the manual and paper work for managing hotels and guest's data.

Through this project, we aim to design, analysis, implement (using C#), and test a simple hotel management system that fulfils basic functionalities for two primary types of users: **guests** and **manager**.

The system's database maintains **I/O files** (not temporary lists or arrays) for all involved guests, rooms, reservations, payments, and services. This information must be permanent and is accessed by different users to achieve their desired transactions and be updated continuously throughout the running time of the system. Every **guest** has the following information: national ID (unique), name, password, phone No, and bank balance. Whereas every **room** has a number (unique), type, price (per day), and availability. Regarding the **reservation**, it has the following information: ID (unique), room number, guest's national ID, check-in date, check-out date, status, and meals. For every **payment** record, the system saves the following information: bill number (unique), guest's national ID, source, amount, and status. Lastly, every **service** has ID (unique), guest's national ID, description, and notes.

The system must provide secure login for both manager and guests. There is only one manager and he uses the following information to login: "**m2024**" as ID and "**00**" as password. On the other hand, every guest uses his national ID and password to login. As a prerequisite, your implementation program must initially load the I/O files with a set of guests and rooms **<u>at the beginning of first run</u>** and then reflect any changes in future on these files. The following data must be loaded **<u>as is</u>** upon running your program for the **<u>first time</u>**:

[**GUESTS**]

National ID	Name	Password	Phone No	Bank Balance
12345	Omar	11	05215	450
12546	Khaled	22	01459	550
16556	Salma	33	04122	660
18730	Ahmad	44	02250	720

[**ROOMS**]

Number	Type	Price per day	Available?
442	single	25	True
102	double	30	True
506	double	32	False
702	suite	40	True
333	double	34	True

Requirement Specifications

1. System must check the correctness of login information (ID and password) for both types of users and display an error message when one or both are incorrect preventing the user from logging in the system.

2. The rooms in the hotel may be one of the following types: single, double, or suite.

3. Regarding the reservation, the following requirements must be satisfied:
   - reservation ID must be a 4-digit number.
   - Status may be one of the following values:
     confirmed --> after the guest finishes reservation process but before checking in,
     checked-in --> once the guest checks in,
     checked-out --> once the guest checks out.
   - Meals may take three values: [1]breakfast, [2]breakfast and lunch, [3]full board. Based on the selection, reservation amount is affected as follows:
   breakfast --> **amount = number of residence days * room price**,
   breakfast and lunch --> **amount = 1.2 *** (**number of residence days * room price)**,
   full board --> **amount = 1.4 *** (**number of residence days * room price)**
   - The guest may get discount (amount=0.6 * amount) if his reservation's check-in date is one of the following dates: **1-2-2025**, **22-4-2025**, or **10-10-2025**.

4. The system provides two services: car rental and kids zone. The amount of selected service is calculated as follows:
   - the amount of the "car rental" service= **number of rental days * 10**
   - the amount of the "kids zone" service = **number of children * 5**

5. Regarding service record, the following requirements must be satisfied:
   - description field stores the service type: “car rental” or “kids zone”.
   - notes field stores either: “number of rental days = ***n***” (for car rental service),
   or “number of children = ***n***” (for kids zone service).
   n: is an integer number

6. Regarding payment, the following requirements must be satisfied:
   - payment bill may be created for either a reservation or a service.
   - payment status is either "paid" or "not paid". After confirming a reservation or requesting a service, the status is set to “not paid”.
   - the status is changed to “paid” after the guest pays for his reservation or requested services.

7. The system is responsible for generating profit report which calculates **separately** the total income from the following sources:
   - room reservations.
   - kids zone.
   - car rental.

The system is required to satisfy the main functional requirements for all users (manager and guests). This necessitate to explore high-level use cases then decomposing them into sub use cases to draw a full image of the involved implementation steps. Consequently, our system has the following high-level use cases:

Guest functions

1. Reserve a room
2. Request a service
3. Check in
4. Check out
5. Pay for a reservation
6. Pay for a service

Manager functions

1. View all guests
2. View all reservations
3. View all services
4. View all payments
5. View all rooms
6. Update room information
7. Generate profit report

YOUR PROGRAM MUST SHOW ONLY THE FUNCTIONS RELATED TO THE CURRENTLY LOGGED-IN USER. THEREFORE, UPON RUNNING YOUR PROGRAM, THE SYSTEM MUST DISPLAY TWO LOGIN OPTIONS (LOGIN SCREEN): AS [1]GUEST OR AS [2]MANAGER. THEN BASED ON THE SELECTED USER TYPE AND SUCCESSFUL LOGIN, THE SYSTEM MUST DISPLAY THE OPTIONS RELATED TO THE LOGGED-IN USER. THE OPTIONS MENU FOR THE TWO USER TYPES MUST PROVIDE “LOG OUT” OPTION TO RETURN TO THE LOGIN SCREEN TO ENABLE HIM TO LOG IN WITH DIFFERENT USER TYPE.

In details, below are the involved steps for each use case:

Use case 1: Reserve a room

- The system displays all available rooms and asks guest to enter room number.
- The system asks guest to enter both check-in date and check-out date.
- The system displays meals options and asks guest to select desired meal option.
- The system asks guest to enter reservation ID.
- The system creates reservation record and sets status to “confirmed”.
- The system saves the reservation record into database.
- The system creates payment record for the reservation.
- The system asks guest to enter bill number.
- The system sets payment source to “reservation”.
- The system calculates the amount for the reservation based on number of residence days, selected meals, and if check-in date matches one of dates that offers discount on reservation amount.
- The system sets the payment status to “not paid”.
- The system saves the payment record into database.
- The system sets the availability of the involved room to “False”.

Use case 2: Request a service

- The system displays all services types provided for hotel guests (**kids zone** or **car rental**).
- The system asks guest to enter service ID, service description, and a number (represents number of rental days or number of children).
- The system sets “notes” field based on entered values in previous step.
- The system creates service record and saves it into database.
- The system creates payment record for the service.
- The system asks guest to enter bill number.
- The system sets payment source to “kids zone” or “car rental” based on the service description entered previously.
- The system calculates the amount for the service based on service type and notes.
- The system sets the payment status to “not paid”.
- The system saves the payment record into database.

Use case 3: Check in

- The system displays all reservations with status=“confirmed” which belong to currently logged-in guest.
- The system asks guest to enter reservation ID.
- The system changes the status of selected reservation to “checked-in”.
- The system updates the involved reservation in database.

Use case 4: Check out

- The system displays all reservations with status="checked-in" which belong to currently logged-in guest.
- The system asks guest to enter reservation ID.
- The system changes the status of selected reservation to "checked-out".
- The system updates the involved reservation in database.
- The system sets the availability of the involved room to "True"

Use case 5: Pay for a reservation

- The system displays all **unpaid** payment records related to reservations that belong to the currently logged-in guest.
- The system asks guest to enter bill number.
- The system updates the bank balance of the currently logged-in guest based on the payment amount.
- The system sets the payment status to "paid".
- The system updates the involved payment and guest records into database.

Use case 6: Pay for a service

- The system displays all **unpaid** payment records related to services (car rental or kids zone) that belong to the currently logged-in guest.
- The system asks guest to enter bill number.
- The system updates the bank balance of the currently logged-in guest based on the payment amount.
- The system sets the payment status to "paid".
- The system updates the involved payment and guest records into database.

Use case 7: View all guests

- The system displays full information of all saved guests in database.

Use case 8: View all reservations

- The system displays full information of all saved reservations in database.

Use case 9: View all services

- The system displays full information of all saved services in database.

Use case 10: View all payments

- The system displays full information of all saved payments in database.

Use case 11: View all rooms

- The system displays full information of all saved rooms in database.

Use case 12: Update room information

- The system displays all available rooms and prompts the manager to enter room number to update.
- The system asks manager to enter new values for “type” and “price” fields.
- The system updates information of the involved room in database.

Use case 13: Generate profit report

- The system calculates and displays separately the total amount gained from the following sources:
  - Room reservations.
  - Car rental.
  - Kids zone.
- This is performed by traversing all “**paid**” payments in database and checking “source” field to determine the biller, then finding the summation of all amounts for each source.

You are required to satisfy and implement every step above, else your grade will be affected accordingly. Please don’t assume anything even if it makes sense. In case you do not understand something, please feel free to ask about it.